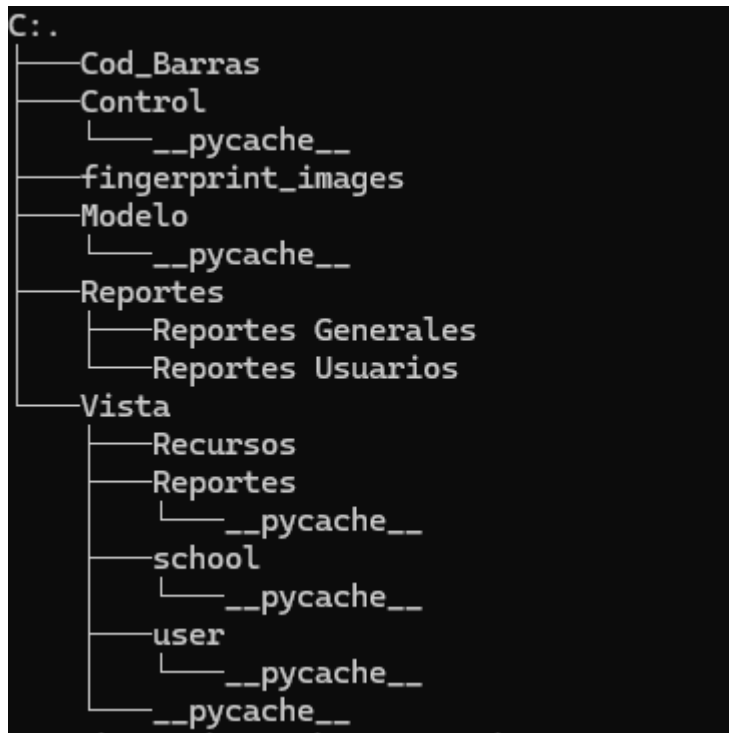


Manual del desarrollador

Control Reloj

Carpetas del sistema:



Img.01: Carpetas en el sistema.

- Cod_Barras: Comprende a la carpeta en la cual se almacenarán las imágenes de código de barras del sistema.
- Control: Aquí se encuentra la carpeta que almacena a todos los controladores del sistema.
- fingerprint_images: Almacena las imágenes si es necesario su depuración de las huellas de las personas.
- Modelo: Aquí se encuentra la carpeta que almacena todos los Modelos del sistema.
- Reportes: Aquí están las carpetas de reporte tanto generales donde se puede apreciar de manera general las personas, como el de reporte de usuarios que es ya un reporte de manera más individual.
- Vista, esta posee subcarpetas según la interfaz que se requiera mostrar, por ejemplo tenemos:
 - Recursos, por ahora es para dejar los recursos como imágenes a utilizar,
 - Reportes, aquí se encuentran las interfaces de reporte tanto individuales como generales.
 - School, básicamente la interfaz relacionada a los CRUD de los establecimientos.
 - User, este al igual que la de School posee todo lo relacionado de los CRUD de los usuarios incluyendo la interfaz para modificar una asistencia.

Clases y funciones del sistema:

	Nombre del archivo:	Modelo.DatabaseConectio n.py
Clase:	ConexionMySQL	
Definición:		
Clase que se encarga conectar con la base de datos, en este caso con la de mysql usando el mysql.connector.		
• Funciones:		
ejecutar_consulta	Ejecuta una consulta SELECT y devuelve los resultados	
ejecutar_vista	Ejecuta una vista y devuelve los resultados	
ejecutar_procedimiento	Ejecuta un procedimiento almacenado y devuelve los resultados	
ejecutar_funcion	Ejecuta una función de MySQL y devuelve el resultado	
ejecutar_procedimiento_ins ert	Ejecuta un procedimiento almacenado tipo INSERT con parámetros	
ejecutar_procedimiento_upd ate	Ejecuta un procedimiento almacenado tipo UPDATE con parámetros	
ejecutar_procedimiento2	Ejecuta un procedimiento almacenado y devuelve los resultados	
ejecutar_procedimiento_Re gistrar_Evento	Ejecuta un procedimiento almacenado tipo INSERT con parámetros y devuelve un valor booleano	
fetch_biometric_models_bat ch	Recupera un lote de modelos biométricos junto con los nombres de la tabla usuarios	
insertar_evento	Inserta un nuevo evento en la tabla eventos.	
modificar_evento	Modifica la hora de un evento existente donde coincidan idregistro y tipo_evento.	
cerrar	Cierra la conexión	

	Nombre del archivo:	Modelo.barra.py
Clase:	Lectora_codigo_barras	
Definición:		

Clase que se encarga de homogeneizar códigos relacionados con RUNs y otros datos para generar códigos únicos.

- **Funciones:**

homogenizar_run

Función para homogeneizar el RUN.
Si el RUN termina en "K" precedido de un guión,
reemplaza el guion por "0"
y elimina puntos y guiones del RUN.

Args:

run (str): RUN chileno a homogeneizar.

Returns:

str: RUN limpio sin puntos, guiones y con "K"
reemplazado por "0" si es necesario.

homogenizar_cadena

Función para homogeneizar la cadena de texto de entre 4
y 7 caracteres con guión.
Asegura que la cadena tenga entre 4 y 7 caracteres y no
tenga guiones ni puntos.

Args:

cadena (str): Cadena de texto a homogeneizar.

Returns:

str: Cadena limpia sin guiones ni puntos.

Raises:

ValueError: Si la cadena no tiene entre 4 y 7
caracteres.

generar_codigo_unico

Función para generar un código único a partir de los
códigos de escuela y persona.
La entrada se concatena, se genera un hash SHA-256 y se
reduce a 12 dígitos únicos.

Args:

codigo_escuela (str): Código de la escuela.

codigo_persona (str): Código de la persona (RUN).

Returns:

str: Un código único de 12 dígitos.

generar_codigo_barras

Genera un código de barras EAN-13 y lo guarda como
imagen en la carpeta 'Cod_Barras'.

Args:

contenido (str): Número que deseas codificar (debe
tener 12 dígitos para EAN-13).

archivo_salida (str): Nombre del archivo de salida
sin extensión.

	Returns: str: Código EAN-13 completo (incluye el dígito verificador) si tiene éxito. None: Si ocurre un error.
--	--

	Nombre del archivo:	Modelo.huella.py
Clase:	FingerprintScanner	
Definición:		
Clase que maneja el registro y la lectura de huellas dactilares a través de un lector biométrico. Proporciona métodos para capturar, verificar y almacenar huellas.		
• Funciones:		
__init__	Constructor de la clase FingerprintScanner. Inicializa el lector de huellas y establece los valores predeterminados de la clase.	
initialize_zkfp2	Inicializa el lector de huellas.	
generar_codigo_unico	Función para generar un código único a partir de los códigos de escuela y persona. La entrada se concatena, se genera un hash SHA-256 y se reduce a 12 dígitos únicos. Args: codigo_escuela (str): Código de la escuela. codigo_persona (str): Código de la persona (RUN). Returns: str: Un código único de 12 dígitos.	
save_fingerprint_image_locally	Guarda una imagen de huella dactilar utilizando la biblioteca Pillow. :param image_data: Los datos de la imagen en formato bytes. :param width: El ancho de la imagen (por defecto, 256). :param height: La altura de la imagen (por defecto, 288). :param directory: La ruta donde se guardarán las imágenes (por defecto, 'fingerprint_images'). :return: La ruta completa del archivo guardado.	
capture_handler	Procesa la captura de huellas para registro.	

read_handler	Procesa la captura de huellas para lectura.
process_fingerprint	Procesa una huella dactilar y la guarda si es necesario.
save_combined_fingerprint	Combina las huellas capturadas y las guarda en la base de datos.
start_registration	Inicia el proceso de registro de huellas.
start_reading	Inicia el proceso de lectura de huellas.
listen_to_fingerprints	Inicia la escucha para capturar huellas.
shutdown	Cierra todos los recursos.
mostrar_mensaje_info	Muestra un mensaje de información en la interfaz gráfica.
GetRute	Obtiene la ruta de la imagen de huella capturada.
getHuella	Obtiene el modelo biométrico de la huella capturada.
getInicializado	Verifica si el lector de huellas ha sido inicializado correctamente.
setRBD	Establece el RBD para la verificación.
setMarcacion	Establece el tipo de marcación para el registro de asistencia.
verify_fingerprint	Comparar una huella tomada en el momento con las almacenadas en la base de datos y devolver el nombre de la persona.

	Nombre del archivo:	Modelo.pistola.py
Clase:	Lectora_codigo_barras(QObject)	
Definición:		
Clase que maneja la lectura de códigos de barras a través de un lector y emite señales cuando la lectura se ha completado. También gestiona el tipo de marcación y el temporizador para finalizar la lectura después de un tiempo de inactividad.		
• Funciones:		
__init__	Constructor de la clase Lectora_codigo_barras. Inicializa las variables necesarias para manejar la lectura del código de barras.	
reset_timer	Resetea el temporizador cada vez que se recibe un nuevo carácter. Cancela el temporizador anterior y crea uno nuevo que finalizará la lectura si no se recibe un nuevo carácter en el tiempo establecido.	

set_tipo_marcacion	Permite actualizar el tipo de marcación desde la UI. Este método establece el tipo de marcación que se utilizará al finalizar la lectura del código.
finalizar_lectura	Método que se llama cuando el tiempo de espera se agota, es decir, cuando no se recibe un nuevo carácter en el tiempo configurado por el temporizador. Se emite una señal con el código leído y el tipo de marcación.
detener_lectura	Detiene el listener de teclado manualmente, lo que interrumpe el proceso de lectura.
on_press	Captura la tecla presionada y agrega el carácter al código leído. Si se presiona la tecla Enter, finaliza la lectura.
leer_codigo_barras	Lee el dato ingresado por la pistola de códigos de barras y lo muestra en pantalla. Inicia el listener de teclado para capturar las teclas presionadas

	Nombre del archivo:	Modelo.reg_grupal.py
Clase:	R_Grupal	
Definición:		
Clase que representa un registro grupal. Hereda de la clase `Registro` y añade propiedades adicionales para gestionar los datos de los usuarios.		
• Funciones:		
__init__	Constructor que inicializa un registro grupal con los detalles del grupo, incluyendo el RUN, estado de completado, nombre, apellido y cantidad de asistidos. :param run: RUN del registro (identificador único). :param completo: Indica si el registro está completo o no. :param nombre: Nombre de la persona registrada. :param apellido: Apellido de la persona registrada. :param asistidos: Numero de días asistidos	
Getters and Setters	Funciones que permiten establecer y devolver los parametros o atributos que usa la clase	

	Nombre del archivo:	Modelo.reg_individual.py
Clase:	R_individual	
Definición:		
Clase que representa un registro individual de asistencia. Hereda de la clase `Registro` y contiene atributos específicos para gestionar la entrada y salida de una persona, incluyendo las horas de entrada, salida y colación, así como el identificador de tarjeta		
• Funciones:		
__init__	Constructor que inicializa un registro individual con los detalles de la persona registrada, incluyendo su hora de entrada, salida, y las horas de colación. :param run: RUN del registro (identificador único). :param completo: Indica si el registro está completo o no. :param rbd: Identificador del establecimiento o curso. :param fecha: Fecha del registro. :param h_entrada: Hora de entrada. :param h_salida: Hora de salida. :param hc_salida: Hora de salida para colación. :param hc_entrada: Hora de entrada de colación. :param idTarjeta: (Opcional) Identificador de la tarjeta asociada. Si no se proporciona, se asigna vacío.	
Getters and Setters	Funciones que permiten establecer y devolver los parámetros o atributos que usa la clase	

	Nombre del archivo:	Modelo.registro.py
Clase:	Registro	
Definición:		
Clase base que representa un registro con un identificador único (RUN) y un estado de completitud. Se utiliza para representar los registros básicos de una persona o entidad, y su estado de completitud.		
• Funciones:		
__init__	Inicializa un registro con un RUN (identificador único) y un estado de completitud. :param run: Identificador único del registro. :param completo: Estado del registro, indica si está completo o no.	

Getters and Setters	Funciones que permiten establecer y devolver los parámetros o atributos que usa la clase
---------------------	--

	Nombre del archivo:	Modelo.tipo_usuario.py
Clase:	Tipo_user	
Definición:		
Clase que representa un tipo de usuario en el sistema, con un identificador y una descripción. Esta clase permite acceder y modificar el id del tipo de usuario y su descripción.		
• Funciones:		
__init__	Inicializa un tipo de usuario con un identificador y una descripción. :param id: Identificador único del tipo de usuario. :param descripcion: Descripción detallada del tipo de usuario.	
Getters and Setters	Funciones que permiten establecer y devolver los parámetros o atributos que usa la clase	

	Nombre del archivo:	Modelo.user.py
Clase:	User	
Definición:		
Clase que representa un usuario en el sistema con su información personal, tipo de usuario, clave y huella dactilar. Esta clase permite acceder y modificar los detalles de un usuario, incluyendo su huella.		
• Funciones:		
__init__	Inicializa un objeto User con los detalles personales del usuario, el tipo de usuario y la clave. :param run: Identificador único del usuario (generalmente RUT en algunos países). :param nombre: Nombre del usuario. :param apellido_1: Primer apellido del usuario. :param apellido_2: Segundo apellido del usuario. :param id_tipo_usuario: Identificador que define el tipo de usuario. :param clave: Clave de acceso del usuario. :param huella: Huella digital del usuario, se define como un bytearray vacío si no se proporciona.	

Getters and Setters	Funciones que permiten establecer y devolver los parámetros o atributos que usa la clase
---------------------	--

	Nombre del archivo:	Control.C_Registro.py
Clase:	C_Registro	
Definición:		
Clase encargada de gestionar los registros de asistencia y reportes de usuarios. Proporciona métodos para generar reportes individuales y generales, verificar usuarios, y registrar asistencias en la base de datos.		
• Funciones:		
Reporte_Usuario	Genera un reporte individual de asistencia para un usuario específico. Parámetros: - rbd (str): Identificador del establecimiento. - run (str): RUN del usuario. - fecha_ini (str): Fecha de inicio del reporte. - fecha_fin (str): Fecha de fin del reporte. - mes (str): Mes del reporte. - opcion (int): Opción para filtrar el reporte. Retorna: - list: Lista de datos con la información de asistencia del usuario.	
Reporte_General	Genera un reporte general de asistencia para todos los usuarios de un establecimiento. Parámetros: - rbd (str): Identificador del establecimiento. - fecha_ini (str): Fecha de inicio del reporte. - fecha_fin (str): Fecha de fin del reporte. - mes (str): Mes del reporte. - year (str): Año del reporte. - opcion (int): Opción para filtrar el reporte. Retorna: - list: Lista de datos con la información de asistencia general.	
Obtener_tarjeta	Obtiene el ID de la tarjeta asociada a un usuario y establecimiento. Parámetros: - run (str): RUN del usuario. - rbd (str): Identificador del establecimiento.	

	Retorna: - str: ID de la tarjeta asociada al usuario.
Verificar_usuario_clave	Verifica las credenciales de un usuario y registra su asistencia si son válidas. Parámetros: - run (str): RUN del usuario. - clave (str): Clave del usuario. - rbd (str): Identificador del establecimiento. - tipo_E (str): Tipo de evento a registrar. Retorna: - bool: True si la asistencia se registró correctamente, False en caso contrario.
Registrar_Asistencia	Registra la asistencia de un usuario en la base de datos. Parámetros: - idTarjeta (str): ID de la tarjeta del usuario. - tipo_E (str): Tipo de evento a registrar. Retorna: - bool: True si el registro fue exitoso, False en caso contrario.

	Nombre del archivo:	Control.C_School.py
Clase:	C_Escuela	
Definición:		
Clase encargada de gestionar las operaciones relacionadas con las escuelas. Permite agregar, modificar, eliminar y buscar escuelas en la base de datos.		
Funciones:		
agregar_escuela	Agrega una nueva escuela a la base de datos utilizando el RBD y el nombre proporcionados. Parámetros: rbd (int): El RBD de la escuela a agregar. nombre (str): El nombre de la escuela a agregar. Retorna: bool: True si la operación fue exitosa, False en caso contrario.	
modificar_escuela	Modifica los datos de una escuela en la base de datos utilizando el RBD y el nuevo nombre proporcionados.	

	Parámetros: rbd (int): El RBD de la escuela a modificar. nombre (str): El nuevo nombre de la escuela. Retorna: bool: True si la operación fue exitosa, False en caso contrario.
eliminar_escuela	Elimina una escuela de la base de datos utilizando el RBD proporcionado. Parámetros: rbd (int): El RBD de la escuela a eliminar. Retorna: bool: True si la operación fue exitosa, False en caso contrario.
buscar_escuela	Busca una escuela en la base de datos utilizando el RBD proporcionado. Parámetros: rbd (int): El RBD de la escuela a buscar. Retorna: str: El nombre de la escuela si se encuentra, una cadena vacía si no.
text_mayuscula	Convierte el texto proporcionado a mayúsculas. Parámetros: Texto (str): El texto que se desea convertir a mayúsculas. Retorna: str: El texto en mayúsculas.

	Nombre del archivo:	Control.C_tipo_user.py
Clase:	C_tipo_usuario	
Definición:		
Clase que gestiona el acceso y manipulación de los tipos de usuarios en el sistema. Carga los tipos de usuarios desde la base de datos y proporciona los datos necesarios.		
Funciones:		
cargar_lista	Carga la lista de tipos de usuario desde la base de datos. Realiza una consulta a la vista 'vista_tipo_usuario' en la base de datos y	

	retorna una lista de objetos Tipo_user, que contienen el ID y la descripción de cada tipo de usuario. Retorna: list: Lista de objetos Tipo_user.
--	--

	Nombre del archivo:	Control.C_usuario.py
Clase:	C_user	
Definición:		
Clase encargada de gestionar las operaciones relacionadas con los usuarios. Permite ingresar, modificar, eliminar, buscar usuarios y manejar su autenticación y tarjetas asociadas.		
• Funciones:		
Ingresar_Usuario	Ingresar un nuevo usuario en la base de datos con los datos proporcionados. Parámetros: RUN (str): El RUN del usuario a ingresar. NOMBRE (str): El nombre del usuario. APELLIDO_1 (str): El primer apellido del usuario. APELLIDO_2 (str): El segundo apellido del usuario. TIPO_USER (str): El tipo de usuario (por ejemplo, administrador, empleado). CLAVE (str): La contraseña del usuario. HUELLA (str, opcional): La huella del usuario, si aplica.	
Ingresar_Tarjeta	Ingresar una tarjeta asociada a un usuario y escuela en la base de datos. Parámetros: RUN (str): El RUN del usuario. RBD (int): El RBD de la escuela. COD_TARJETA (str): El código de la tarjeta.	
Ver_Usuario	Busca un usuario en la base de datos utilizando su RUN. Parámetros: run (str): El RUN del usuario a buscar. Retorna: usuario (User): Un objeto User con los datos del usuario encontrado, o None si no se encuentra.	
Modificar_Usuario	Modifica los datos de un usuario en la base de datos con los nuevos valores proporcionados. Parámetros:	

	RUN (str): El RUN del usuario a modificar. NOMBRE (str): El nuevo nombre del usuario. APELLIDO_1 (str): El nuevo primer apellido del usuario. APELLIDO_2 (str): El nuevo segundo apellido del usuario. TIPO_USER (str): El nuevo tipo de usuario. CLAVE (str): La nueva contraseña del usuario.
Eliminar_Usuario	Elimina un usuario de la base de datos utilizando su RUN. Parámetros: RUN (str): El RUN del usuario a eliminar.
text_mayuscula	Convierte el texto proporcionado a mayúsculas. Parámetros: Texto (str): El texto que se desea convertir a mayúsculas. Retorna: str: El texto en mayúsculas.
verificar_formato_run	Verifica si el RUN proporcionado está en el formato XX.XXX.XXX-X o X.XXX.XXX-X. Parámetros: run (str): El RUN a verificar. Retorna: bool: True si el formato es válido, False en caso contrario.
formatear_run	Formatea el RUN proporcionado eliminando puntos y guiones, y devolviendo el formato XX.XXX.XXX-X o X.XXX.XXX-X. Parámetros: run (str): El RUN a formatear. Retorna: str: El RUN formateado o None si el formato no es válido.
validar_run	Valida si el RUN proporcionado es correcto según el algoritmo del dígito verificador. Parámetros: run (str): El RUN a validar. Retorna: bool: True si el RUN es válido, False en caso contrario.

Obtener_tarjeta	Obtiene el ID de la tarjeta asociada a un usuario y una escuela. Parámetros: run (str): El RUN del usuario. rbd (int): El RBD de la escuela. Retorna: str: El ID de la tarjeta asociada, o una cadena vacía si no se encuentra.
Ver_Eventos_Individual	Obtiene los eventos individuales asociados a una tarjeta, en una fecha y tipo de evento específicos. Parámetros: fecha (str): La fecha de los eventos a consultar. idtarjeta (str): El ID de la tarjeta asociada. tipo_evento (str): El tipo de evento a buscar. Retorna: tuple: El ID del registro y la hora del evento, o cadenas vacías si no se encuentra.
InsertarEvento	Inserta un nuevo evento en la base de datos. Parámetros: idregistro (str): El ID del registro del evento. tipo (str): El tipo de evento. hora (str): La hora del evento. Retorna: bool: True si la inserción fue exitosa, False en caso contrario.
ModificarEvento	Modifica un evento existente en la base de datos. Parámetros: idregistro (str): El ID del registro del evento. tipo (str): El nuevo tipo de evento. hora (str): La nueva hora del evento. Retorna: bool: True si la modificación fue exitosa, False en caso contrario.
verificar_usuario	Verifica las credenciales de un usuario (RUN y clave) en la base de datos. Parámetros: run (str): El RUN del usuario. clave (str): La clave del usuario. Retorna: str: El tipo de usuario si las credenciales son

	correctas, una cadena vacía si no.
encode_password	Genera un hash SHA-256 de la contraseña proporcionada y la codifica en base64. Parámetros: password (str): La contraseña a codificar. Retorna: str: La contraseña codificada en base64.

	Nombre del archivo:	Control.control_lectora.py
Clase:	ControladorLector	
Definición:		
Controlador para la gestión de la lectura y generación de códigos de barras y datos relacionados. Proporciona métodos para homogenizar datos y generar códigos únicos o de barras a través de la lectora.		
• Funciones:		
__init__	Inicializa el controlador del lector de códigos de barras. Crea una instancia de la clase Lectora_codigo_barras.	
homogenizar_run	Homogeniza el RUN (Rol Único Nacional) proporcionado, ajustando el formato. Parámetros: run (str): El RUN a ser homogenizado. Retorna: str: El RUN homogenizado. Excepciones: ValueError: Si ocurre un error al intentar homogenizar el RUN.	
homogenizar_cadena	Homogeniza la cadena de texto proporcionada. Parámetros: cadena (str): La cadena a ser homogenizada. Retorna: str: La cadena homogenizada. Excepciones: ValueError: Si ocurre un error al intentar homogenizar la cadena.	

generar_codigo_unico	Genera un código único utilizando los códigos de la escuela y de la persona. Parámetros: codigo_escuela (str): El código de la escuela. codigo_persona (str): El código de la persona. Retorna: str: El código único generado. Excepciones: ValueError: Si ocurre un error al generar el código único.
generar_codigo_barras	Genera un código de barras a partir del contenido proporcionado y guarda el archivo generado. Parámetros: contenido (str): El contenido para el código de barras. archivo_salida (str): La ruta donde se guardará el archivo del código de barras. Retorna: str: El código completo generado. Excepciones: ValueError: Si ocurre un error al generar el código de barras.

	Nombre del archivo:	Control.control_pistola.py
Clase:	ControladorLector	
Definición:		
Controlador encargado de gestionar la lectura de códigos de barras y el registro de asistencia. Conecta la lectora de códigos de barras con los procesos de registro de asistencia.		
• Funciones:		
__init__	Inicializa el controlador, creando la instancia de la lectora de códigos de barras y conectando el evento de lectura completada a la función Realizar_Registro.	
iniciar_lectura	Inicia la lectura de códigos de barras si no está activamente en proceso de lectura. No recibe parámetros. Inicia un hilo para ejecutar la lectura del código de barras.	

detener_lectura	Detiene la lectura de códigos de barras si está activamente leyendo. No recibe parámetros. Detiene el proceso de lectura de la lectora.
set_tipo_marcacion	Actualiza el tipo de marcación desde la interfaz de usuario. Parámetros: tipo (str): El tipo de marcación que se debe establecer (por ejemplo, asistencia, salida, etc.).
Realizar_Registro	Registra la asistencia de un usuario después de haber leído un código de barras. Parámetros: codigo (str): El código de barras leído, utilizado para identificar al usuario. tipo_marcacion (str): El tipo de marcación de la asistencia (por ejemplo, entrada, salida). Muestra un mensaje indicando si el registro fue exitoso o fallido.

	Nombre del archivo:	Control.ControlHuella.py
Clase:	FingerprintManager	
Definición:		
Clase encargada de gestionar el escaneo y manejo de huellas dactilares. Controla el proceso de escaneo, captura y lectura de huellas mediante un escáner de huellas dactilares. Además, permite gestionar las opciones de registro y verificación de huellas.		
Funciones:		
<code>__init__</code>	Inicializa el gestor de huellas, configurando el escáner de huellas, el temporizador y los valores predeterminados para las opciones de escaneo y el estado de la huella. También se configura la señal para verificar el escáner de huellas en intervalos regulares.	
<code>start_fingerprint_scanning</code>	Inicia el proceso de lectura de huellas. Si el escaneo no está activo, comienza la lectura y se activa el temporizador para comprobar el escáner cada 100 ms.	
<code>stop_fingerprint_scanning</code>	Detiene el proceso de lectura de huellas.	

	Si el escaneo está activo, se detiene el temporizador y el escáner.
check_fingerprint_scanner	Verifica si se ha capturado una huella y maneja el proceso según la opción seleccionada. Si se detecta una huella, se maneja según la opción de registro o lectura establecida.
set_option	Establece la opción de operación para el escaneo de huellas. Parámetros: opcion (int): La opción seleccionada (1 para registro, 2 para lectura).
get_huella	Obtiene la huella capturada. Retorna la huella actual almacenada como un objeto bytearray.
get_inicializado	Verifica si el escáner de huellas está inicializado y listo para usar. Retorna el estado de inicialización del escáner.
setRBD	Establece el RBD (registro de establecimiento) para el escáner. Parámetros: rbd (str): El RBD asociado al escáner.
setMarcacion	Establece el tipo de marcación para el escáner. Parámetros: tipo (str): El tipo de marcación (por ejemplo, asistencia, verificación).
shutdown	Cierra todos los recursos y detiene el escaneo de huellas. Llama al método stop_fingerprint_scanning para detener el escaneo y cierra el escáner.

	Nombre del archivo:	Vista.Reportes.V_ReportG.py
Clase:	ReporteApp	
Definición:		
Clase encargada de gestionar la interfaz para generar reportes generales. Permite al usuario seleccionar el tipo de reporte (semanal, mensual o anual) y generar un archivo Excel..		
Funciones:		
limpiar_fecha	Reemplaza '/' y '-' por '_' en una fecha dada. Parámetros: - fecha: La fecha en formato de cadena (str) que se desea limpiar. Retorna: - Una cadena con los delimitadores '/' y '-' reemplazados por '_'. 	
generar_excel	Genera un archivo Excel con datos de ejemplo según el tipo de reporte especificado. Parámetros: - rbd: El RBD del establecimiento. - tipo_reporte: El tipo de reporte a generar (1: semanal, 2: mensual, 3: anual). - desde: La fecha de inicio (solo para tipo_reporte 1). - hasta: La fecha de fin (solo para tipo_reporte 1). - mes: El mes (solo para tipo_reporte 2). - anio: El año (solo para tipo_reporte 3). Retorna: - El nombre del archivo Excel generado	
__init__	Constructor de la clase. Inicializa la interfaz para seleccionar el tipo de reporte y los campos necesarios según la opción seleccionada. Parámetros: - rbd: El RBD del establecimiento.	
generar_reporte	Maneja la generación del reporte según la selección realizada por el usuario. Dependiendo del tipo de reporte seleccionado (semanal, mensual, anual), llama a la función correspondiente para generar el archivo Excel.	

	Nombre del archivo:	Vista.Reportes.V_ReportI.py
Clase:	ReporteUsuarioApp	
Definición:		
Clase encargada de gestionar la interfaz para generar reportes de usuario. Permite al usuario ingresar el RUN y seleccionar el tipo de reporte (semanal o mensual).		
• Funciones:		
limpiar_fecha	Reemplaza '/' y '-' por '_' en una fecha dada. Parámetros: - fecha: La fecha en formato de cadena (str) que se desea limpiar. Retorna: - Una cadena con los delimitadores '/' y '-' reemplazados por '_'.	
limpiar_run	Elimina los puntos y el guion de un RUN chileno. Parámetros: - run: El RUN del usuario a limpiar. Retorna: - El RUN limpio, sin puntos ni guion.	
generar_excel	Genera un archivo Excel con datos de ejemplo según el tipo de reporte. Parámetros: - run: El RUN del usuario. - rbd: El RBD del establecimiento. - tipo_reporte: El tipo de reporte a generar (1: semanal, 2: mensual). - desde: La fecha de inicio (solo para tipo_reporte 1). - hasta: La fecha de fin (solo para tipo_reporte 1). - mes: El mes (solo para tipo_reporte 2). Retorna: - El nombre del archivo Excel generado.	
__init__	Constructor de la clase. Inicializa la interfaz para ingresar el RUN y seleccionar el tipo de reporte. Parámetros: - rbd: El RBD del establecimiento.	
generar_reporte	Maneja la generación del reporte según la selección del tipo de reporte.	

	Verifica que el RUN esté ingresado y llama a la función para generar el reporte correspondiente.
actualizar_interfaz	Habilita o deshabilita los campos de fecha o mes según el tipo de reporte seleccionado.

	Nombre del archivo:	Vista.school.v_S_school
Clase:	EstablecerEstablecimiento	
Definición:		
Clase encargada de gestionar la ventana para establecer el RBD de un establecimiento. Permite ingresar un RBD y almacenarlo en un archivo binario. Emite una señal con el RBD establecido cuando se guarda correctamente.		
• Funciones:		
<code>__init__</code>	Constructor de la clase. Inicializa la interfaz con un campo para ingresar el RBD y un botón para establecerlo.	
<code>mostrar_mensaje</code>	Valida el RBD ingresado, lo guarda en un archivo binario y muestra un mensaje de éxito o error. Emite una señal con el RBD establecido si la operación es exitosa.	
<code>actualizar_interfaz</code>	Habilita o deshabilita los campos de fecha o mes según el tipo de reporte seleccionado.	

	Nombre del archivo:	Vista.school.v_school
Clase:	EstablecimientoForm	
Definición:		
Clase encargada de gestionar la ventana para realizar operaciones con un establecimiento. Permite agregar, modificar o eliminar un establecimiento según la opción seleccionada.		
Funciones:		
<code>__init__</code>	Constructor de la clase. Inicializa la interfaz con el formulario y los botones según la opción seleccionada. Parámetros: - opcion: Define la operación que se va a realizar (1: Agregar, 2: Modificar, 3: Eliminar).	

BuscarEstablecimiento	Realiza una búsqueda del establecimiento usando el RBD ingresado. Si se encuentra, llena el campo de nombre con el resultado. Si no, muestra un mensaje informando que no se encontró el establecimiento.
CRUD_Establecimiento	Realiza la operación correspondiente (Agregar, Modificar o Eliminar) según la opción seleccionada. Muestra un mensaje indicando si la operación fue exitosa o si hubo un error.
reset_fields	Restablecer los campos y desbloquear el input de RBD.
mostrar_mensaje_info	Muestra un mensaje de información en la interfaz gráfica. Parámetros: - mensaje: El texto que se mostrará en el mensaje de información.

	Nombre del archivo:	Vista.user.registro
Clase:	Formulario	
Definición:		
Formulario para el registro de usuarios. Permite ingresar RUN, RBD, nombre, apellidos, tipo de usuario, clave, huella digital y tarjeta, gestionando su almacenamiento y validación.		
• Funciones:		
mostrar_mensaje	Muestra un mensaje emergente con información relevante al usuario. Parámetros: mensaje (str): Texto que se mostrará en la ventana emergente.	
guardar_datos	Obtiene los datos ingresados en el formulario, valida el RUN, los transforma a mayúsculas y los guarda en la base de datos. También asocia la tarjeta generada al usuario registrado. Parámetros: option (int): Indica si se capturó o no una huella digital.	
generar_tarjeta	Genera un código único de tarjeta utilizando el RUN y el RBD del usuario. Luego, crea un código de barras a partir del código	

	generado y lo asigna al campo correspondiente en la interfaz.
Capturar_Huella	Inicia el proceso de captura de huella digital utilizando el lector biométrico. Cambia el estado del lector para que comience el escaneo y almacena la huella.
closeEvent	Se ejecuta cuando la ventana del formulario es cerrada. Desactiva el lector de huellas y muestra un mensaje informativo al usuario. Parámetros: event (QCloseEvent): Evento de cierre de la ventana.
limpiar_run	Elimina los caracteres especiales (puntos y guion) del RUN para su procesamiento. Asegura que el RUN esté en un formato limpio antes de ser almacenado. Parámetros: run (str): RUN ingresado por el usuario. Retorna: str: RUN sin puntos ni guion.
close_window	Cierra la ventana del formulario sin guardar cambios.

	Nombre del archivo:	Vista.user.v_D_userI
Clase:	DeleteUsuario	
Definición:		
Ventana de diálogo para dar de baja a un usuario. Permite buscar un usuario por RUN y eliminarlo del sistema.		
Funciones:		
buscar_run	Busca un usuario en la base de datos a partir del RUN ingresado. Si el usuario existe, carga sus datos en los campos correspondientes y deshabilita la edición del RUN y RBD.	
Eliminar	Elimina (logica) un usuario de la base de datos utilizando el RUN ingresado. Si la eliminación es exitosa, muestra un mensaje de confirmación; en caso de error, muestra un mensaje con la	

	descripción del problema.
cancelar	Restablece el formulario a su estado inicial, permitiendo la edición del RUN. Limpia todos los campos excepto el RBD, que permanece inmutable.
mostrar_mensaje_info	Muestra un cuadro de diálogo con un mensaje de información. Parámetros: mensaje (str): El mensaje que se mostrará en la ventana emergente.

	Nombre del archivo:	Vista.user.v_U_user
Clase:	UpdateUsuario	
Definición:		
Ventana de diálogo para modificar la información de un usuario. Permite buscar un usuario por RUN, actualizar sus datos y guardar los cambios.		
Funciones:		
buscar_run	Busca un usuario en la base de datos a partir del RUN ingresado. Si el usuario existe, carga sus datos en los campos correspondientes y habilita su edición para la modificación.	
cancelar	Restaura el formulario a su estado inicial, permitiendo la edición del RUN. Limpia todos los campos excepto el RBD, que permanece inmutable.	
Modificar	Modifica los datos del usuario en la base de datos con la información ingresada. Si la modificación es exitosa, muestra un mensaje de confirmación; en caso de error, muestra un mensaje con la descripción del problema.	
mostrar_mensaje_info	Muestra un cuadro de diálogo con un mensaje de información. Parámetros: mensaje (str): El mensaje que se mostrará en la ventana emergente.	

	Nombre del archivo:	Vista.user.v_UA_user
Clase:	Up_AsiistenciaForm	
Definición:		
Ventana de diálogo para revisar y modificar la asistencia de un usuario. Permite buscar registros de asistencia por RUN y fecha, y modificar la hora de entrada o salida.		
• Funciones:		
__init__	Inicializa la ventana de revisión de asistencia. Parámetros: - rbd (str): Identificador único de la institución o entidad relacionada.	
bloquear_campos	Bloquea o desbloquea los campos del formulario según el parámetro 'bloquear'. Parámetros: - bloquear (bool): Si es True, bloquea los campos RUN, Fecha y Combobox. Si es False, habilita el campo Hora.	
buscar_asistencia	Busca un registro de asistencia en la base de datos utilizando el RUN, la fecha y el tipo de marcación. Si se encuentra el registro, carga la hora correspondiente en el campo Hora. Si no se encuentra, habilita la inserción de un nuevo registro.	
cambiar_asistencia	Modifica o inserta un registro de asistencia en la base de datos. Dependiendo del valor de 'Insertar', realiza una inserción o una modificación. Muestra un mensaje de éxito o error según el resultado de la operación.	
limpiar_campos	Limpia todos los campos del formulario y restablece el Combobox a su valor predeterminado.	
cancelar_asistencia	Cancela la operación actual, bloquea los campos y limpia el formulario.	
convertir_fecha	Convierte una fecha en formato dd-mm-aaaa o dd/mm/aaaa al formato aaaa-mm-dd. Parámetros: - fecha (str): Fecha en formato dd-mm-aaaa o dd/mm/aaaa. Retorna:	

	- str: Fecha en formato aaaa-mm-dd, o una cadena vacía si el formato es inválido.
mostrar_mensaje_info	Muestra un cuadro de diálogo con un mensaje de información. Parámetros: mensaje (str): El mensaje que se mostrará en la ventana emergente.

	Nombre del archivo:	Vista.actualizarH
Clase:	ActualizadorHora	
Definición:		
Clase encargada de actualizar la hora y la fecha actual de manera continua. Emite señales con la hora y la fecha actualizadas cada segundo		
Funciones:		
run	Método principal que ejecuta un bucle infinito para actualizar la hora y la fecha cada segundo. Emite las señales correspondientes para la hora y la fecha actualizadas.	

	Nombre del archivo:	Vista.login
Clase:	LoginForm	
Definición:		
Clase encargada de gestionar el formulario de inicio de sesión. Permite al usuario ingresar su RUN y clave para autenticarse. Los datos ingresados se validan y si son correctos, la ventana se cierra		
Funciones:		
<code>__init__</code>	Constructor de la clase. Inicializa la interfaz de inicio de sesión con los campos para ingresar RUN y clave, y el botón para intentar ingresar. Parámetros: - rbd: Identificador único de la institución o usuario. - tipo_entrada: Tipo de acceso que se va a realizar.	
Marcaje	Obtiene los datos de los inputs del formulario y los valida. Si el usuario y clave son correctos, cierra la ventana de inicio de sesión.	

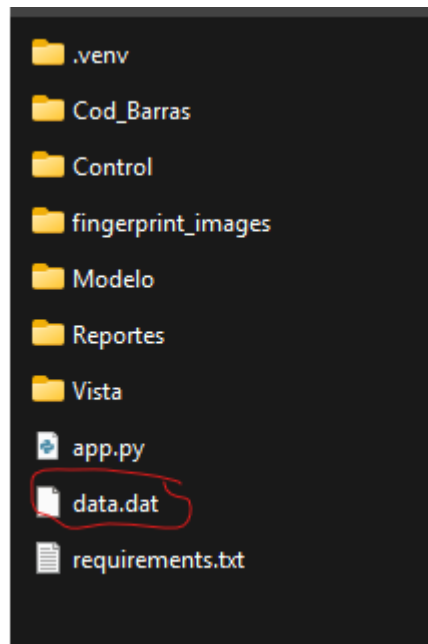
	Nombre del archivo:	Vista.sesionini
Clase:	IniciarSesion	
Definición:		
Clase encargada de gestionar la ventana de inicio de sesión. Permite al usuario ingresar su RUT y clave para autenticar su acceso. Emite una señal cuando el inicio de sesión es exitoso, pasando el rol del usuario.		
● Funciones:		
__init__	Constructor de la clase. Inicializa los elementos de la interfaz gráfica, como los campos de RUT y clave, y el botón de entrada.	
show_alert	Método que valida el RUT y la clave ingresados por el usuario. Si son correctos, emite la señal de inicio de sesión exitoso con el rol del usuario. Si son incorrectos, muestra un mensaje de alerta indicando el error.	
mostrar_mensaje	Muestra un cuadro de diálogo con un mensaje de información. Parámetros: mensaje (str): El mensaje que se mostrará en la ventana emergente.	

	Nombre del archivo:	Vista.inicio
Clase:	MainWindow	
Definición:		
Clase principal que gestiona la interfaz del sistema de control de acceso. Proporciona funcionalidades como autenticación de usuario, selección de tipo de marcación, y administración de usuarios y establecimientos.		
Funciones:		
<code>__init__</code>	Inicializa la ventana principal con la configuración de la interfaz y la hora.	
<code>init_ui</code>	Configura la interfaz de usuario, incluyendo etiquetas de hora y fecha, botones de autenticación y selección de tipo de marcación.	

crear_menu	Crea y configura la barra de menú con las opciones principales.
menu_inicio	Configura el menú de inicio con opciones de sesión y establecimiento.
menu_Usuarios	Configura el menú de Usuario con opciones CRUD del personal Visible: Admin, Director y encargados.
menu_Establecimientos	Configura el menú de establecimiento con opciones CRUD de estos mismos SOLO VISIBLE ADMIN.
menu_Report	Configura el menú de reportes con opciones de visualicion de reportes. Visibilidad Admin, director y encargados
salir	Al momento de salir inicia si realmente quiere salir
open_child_window	Abre el inicio de sesión para administrador, director y encargados
Set_Establecimiento	Método que establece que institución está usando esta interfaz
open_Login_window	Abre el inicio de sesion para todo usuario
usersControl	Segun el menú que seleccione será reenviado
schoolControl	Segun el menú que seleccione será reenviado a una u otra vista del CRUD del establecimiento
reportControl	Segun el menú que seleccione será reenviado a una u otra vista de los reportes
mostrarOcultos	Según el tipo de usuario, se mostrará y ocultarán elementos del menú.
update_hora	Actualizar la hora en el QLabel
update_fecha	Actualizar la fecha en el QLabel
on_method_changed	Método que se ejecuta cuando se cambia el método de identificación.
on_marcacion_changed	Se ejecuta cuando cambia el tipo de marcación
closeEvent	Método que se ejecuta al cerrar la ventana para detener la detección de huellas.
leer_Establecimiento	Lee el establecimiento desde un archivo binario y lo devuelve.
Cambiar_Title	Modifica el título de la ventana principal

Otros:

Recuerda que importante ESTABLECER la institución para que todo funcione correctamente y se encontrará en el siguiente archivo:



img02. Archivo de establecimiento del colegio.